Flutter Developer Assignment

Objective:

Develop a Flutter application that allows users to view a paginated list of users fetched from an API, add new users with offline support, and navigate to a movie listing screen. The movie list should also support pagination and navigation to a movie detail screen. The app should adhere to best practices, including state management, dependency injection, and asynchronous programming using Streams or Riverpod.

Assignment Requirements:

1. User List Screen:

- Fetch and display a paginated list of users from the API:
  - **GET** `https://reqres.in/api/users?page={page}`
- Each item should display the **first name**, **last name**, and **avatar image**.
- Implement pagination to load more users as the user scrolls.
- Clicking on a user should navigate to the **Movie List Screen**.

NOTE: Find the API Key at : reqres

2. Add User Functionality:

- Provide an option (e.g., a floating action button) to navigate to a screen where users can create a new user.
- The creation screen should prompt the user to input a **name** and **job**.
- If the device is online, the new user should be immediately posted to the API:
  - **POST** `https://reqres.in/api/users`

Request JSON:

```
{

  "name": "morpheus",

  "job": "leader"

}
```

- If the device is offline, the user data should be stored in **Drift (SQLite) or Hive**.
- When the device regains internet connectivity, the app should automatically sync the offline data using **WorkManager (via Flutter WorkManager plugin)**.
- After successful syncing, update the **ID** of the user in the local database.

3. Offline Bookmarking Feature:

- Implement a feature allowing a user to "Bookmark" or "Favorite" specific movies from the Movie List or Movie Detail screens.
- Every bookmarked movie must be explicitly linked to a specific user created in the app.
- This bookmarking functionality must be fully operational even when the device is completely offline.
- **Crucial Requirement:** A candidate must be able to create a brand new user offline, immediately navigate to the movie list, and bookmark movies for that newly created offline user.
- When internet connectivity is restored, all pending data (both the new users and their associated movie bookmarks) must automatically and successfully sync to the server via WorkManager without data loss or relationship errors.

4. Movie List Screen:

- When a user clicks on any item from the **User List Screen**, navigate to the **Movie List Screen**.
- Fetch a paginated list of trending movies from the API:
  - **GET** `https://api.themoviedb.org/3/trending/movie/day?language=en-US&page={page}&api_key=YOUR_API_KEY`
- Display each movie's **poster image, title, and release date**.
- Implement pagination to load more movies as the user scrolls.
- Clicking on a movie should navigate to the **Movie Detail Screen**.

5. Movie Detail Screen:

- When a user clicks on a movie from the **Movie List Screen**, navigate to the **Movie Detail Screen**.
- Fetch the movie details from the API:
  - **GET** `https://api.themoviedb.org/3/movie/{movie_id}?api_key=YOUR_API_KEY`
- Display **movie title, description, release date, and poster image**.
- Use `http://image.tmdb.org/t/p/w185/{poster_path}` to load movie images.

Alternative Movie APIs (If TMDB is not accessible):

1. **Movie List :** `https://www.omdbapi.com/?s={search_query}&page=2&apikey=<YOUR_API_KEY>`

2. **Movie Detail :** `https://www.omdbapi.com/?apikey=<YOUR_API_KEY>&i={imdbId}`

6. Network Resilience & Error Handling:

- To test application stability, implement a **custom network interceptor** for all API calls (using Dio or the http package).
- This interceptor must purposefully and randomly fail 30% of all **GET** requests by throwing a simulated **SocketException** or returning a 500 Internal Server Error.
- Implement an automatic, silent retry mechanism with exponential backoff for these failed requests.
- The UI—particularly the infinite scrolling/paginated lists —must gracefully handle these failures **without crashing, duplicating data, or flashing intrusive error screens**. Instead, it should display a subtle, non-blocking "reconnecting..." indicator until the background retry succeeds.

Technical Requirements:

- **Programming Language:** Dart
- **Architecture:** Provider, Riverpod, Bloc, or Cubit
- **Dependency Injection:** get_it or Riverpod
- **Networking:** Dio or http package
- **Local Storage:** Drift or any other SQLite based db
- **Offline Handling:** WorkManager plugin for syncing data when the device regains internet connectivity
- **Pagination:** Infinite scroll or PagedListView
- **Asynchronous Data Handling:** Streams, Riverpod, or Bloc
- **Image Loading:** CachedNetworkImage

Generating TMDB API Key:

To fetch movie details, you need an API key from The Movie Database (TMDB). Follow these steps to generate one:

1. Go to TMDB API and sign in or create an account.
2. Navigate to the API section and request an API key.
3. Use the generated API key in the required API requests by replacing `YOUR_API_KEY` in the assignment.

Generating OMDB API Key:

To fetch movie details, you need an API key from The Movie Database (TMDB). Follow these steps to generate one:

1. Go to OMDB API.
2. Request an API key.
3. Use the generated API key in the required API requests by replacing `YOUR_API_KEY`

in the assignment.

Submission Guidelines:

- Provide a link to a public Git repository containing your project code and APK..
- Include a `README.md` file with:
  - A brief description of the app.
  - Any relevant assumptions or considerations made during development.

Evaluation Criteria:

- Correctness and completeness of implemented features.
- Offline first feature is mandatory.
- Adherence to best practices and architectural patterns.
- Code quality, readability, and documentation.
- Handling of edge cases and error scenarios.
- User experience and UI design considerations.

We look forward to reviewing your implementation. If you have any questions or need further clarifications, please feel free to reach out.